

COA-1

ASSESSMENT TOOL-1

Name: Manne Praneeth kumar

Reg No: 192372310

Subject: CSA1703 - Artificial Intelligence

Question 1: Solve the Water Jug problem (4-gallon and 3-gallon jugs to get exactly 2 gallons in the 4-gallon jug).

Answer

Problem Formulation:

- **State Space:** Represented as an ordered pair (X, Y) where X is the water in the 4-gallon jug ($0 \leq X \leq 4$) and Y is the water in the 3-gallon jug ($0 \leq Y \leq 3$).
- **Initial State:** $(0, 0)$ — Both jugs are empty.
- **Goal State:** $(2, Y)$ or specifically $(2, 0)$ — 4-gallon jug contains exactly 2 gallons.
- **Rules/Actions:** Fill 4G, Fill 3G, Empty 4G, Empty 3G, Pour 4G to 3G, Pour 3G to 4G.

Step-by-Step Derivation:

1. $(0, 0) \rightarrow$ Fill the 3-gallon jug. *State: $(0, 3)$*
2. $(0, 3) \rightarrow$ Pour all 3 gallons into the 4-gallon jug. *State: $(3, 0)$*
3. $(3, 0) \rightarrow$ Fill the 3-gallon jug again. *State: $(3, 3)$*
4. $(3, 3) \rightarrow$ Pour from the 3-gallon jug into the 4-gallon jug until it is full. This takes 1 gallon, leaving 2 gallons in the 3-gallon jug. *State: $(4, 2)$*
5. $(4, 2) \rightarrow$ Empty the 4-gallon jug onto the ground. *State: $(0, 2)$*
6. $(0, 2) \rightarrow$ Pour the 2 gallons from the 3-gallon jug into the 4-gallon jug. *State: $(2, 0)$*
— **Goal Reached!**

Python Implementation

```
def water_jug_bfs(jug1_cap, jug2_cap, target):
    # Queue for BFS storing (state, path)
    queue = [(0, 0), []]
    visited = set([(0, 0)])

    while queue:
        (j1, j2), path = queue.pop(0)

        if j1 == target or j2 == target:
            return path + [(j1, j2)]
```

```

# Possible actions
states = [
    (jug1_cap, j2), # Fill jug 1
    (j1, jug2_cap), # Fill jug 2
    (0, j2),        # Empty jug 1
    (j1, 0),        # Empty jug 2
    # Pour jug 1 to jug 2
    (j1 - min(j1, jug2_cap - j2), j2 + min(j1,
jug2_cap - j2)),
    # Pour jug 2 to jug 1
    (j1 + min(j2, jug1_cap - j1), j2 - min(j2,
jug1_cap - j1))
]

for state in states:
    if state not in visited:
        visited.add(state)
        queue.append((state, path + [(j1, j2)]))
return None

# Find and print the solution path
solution = water_jug_bfs(4, 3, 2)
if solution:
    print("Steps to solve:")
    for step, state in enumerate(solution):
        print(f"Step {step}: 4-Gal Jug = {state[0]}, 3-Gal
Jug = {state[1]}")
else:
    print("No solution found.")

```

Question 2: Mars Rover PEAS Analysis and Exploration.

Answer

- 1. Percepts for the agent:** Visual inputs from cameras (topographical data), spectroscopic sensor readings (chemical composition), GPS/Odometry coordinates (position and orientation), temperature variations, and obstacle proximity data.
- 2. Characterize the operating environment:** The Mars surface is *Partially Observable* (due to dust, rocks blocking vision), *Stochastic* (weather changes, slippage in sand), *Sequential* (current actions impact future locations), *Dynamic* (dust storms, changing solar exposure), *Continuous* (terrain coordinates and angles), and typically *Single-agent*.
- 3. Actions the agent can take:** Drive forward/backward, steer left/right, deploy robotic arm, drill into rocks, collect and store samples, orient solar panels, take photographs, and transmit/receive data to Earth.
- 4. Evaluate the performance:** Maximizing the number of novel scientific samples collected, safe distance traversed without critical damage, energy efficiency (maintaining battery levels), and successful transmission of high-value data back to Earth.
- 5. Suitable Agent Architecture:** A **Utility-based Agent** is the most suitable. A simple goal-based agent is insufficient because multiple goals often conflict (e.g., reaching an interesting rock vs. conserving battery vs. avoiding a steep crater). A utility-based agent evaluates the "happiness" or "scientific value vs. risk" of a state, allowing it to make optimal trade-offs in an uncertain environment.

Python Implementation

```
class MarsRoverUtilityAgent:
    def __init__(self):
        self.battery = 100
        self.samples_collected = 0
        self.location = (0, 0)
        self.state = 'IDLE'

    def percept(self, sensor_data):
        self.terrain = sensor_data.get('terrain')
        self.obstacle = sensor_data.get('obstacle_ahead')
        self.sunlight = sensor_data.get('sunlight_level')

    def calculate_utility(self, action):
        # Simplistic utility function evaluating trade-offs
        if action == "COLLECT" and self.terrain ==
```

```

"high_value":
    return 100 - (100 - self.battery) * 0.5
    elif action == "MOVE_FORWARD" and not
self.obstacle:
    return 50
    elif action == "RECHARGE" and self.sunlight ==
"high":
    return 80 if self.battery < 40 else 10
    return -1 # Invalid or risky action

def act(self):
    actions = ["MOVE_FORWARD", "TURN", "COLLECT",
"RECHARGE"]
    best_action = max(actions, key=lambda a:
self.calculate_utility(a))

    if best_action == "COLLECT":
        self.samples_collected += 1
        self.battery -= 10
        return "Collecting scientific sample..."
    elif best_action == "MOVE_FORWARD":
        self.battery -= 2
        return "Moving forward safely."
    elif best_action == "RECHARGE":
        self.battery = min(100, self.battery + 20)
        return "Recharging solar panels."
    else:
        return "Turning to avoid obstacle."

rover = MarsRoverUtilityAgent()
rover.percept({'terrain': 'high_value', 'obstacle_ahead':
False, 'sunlight_level': 'high'})
print(f"Rover Decision: {rover.act()}")
print(f"Battery Level: {rover.battery}%, Samples:
{rover.samples_collected}")

```

Question 3: 8-Queens problem formulation and search space exploration.

Answer

Problem Formulation Components:

- **Initial State:** An empty 8x8 chessboard with 0 queens placed on it.
- **Actions:** Place a queen in an empty square. (Optimized: Place a queen in the next available row/column where it is not attacked).
- **Transition Model:** Returns a new board configuration with one additional queen placed in the specified position.
- **Goal Test:** 8 queens are on the board, and no two queens attack each other (no queens share the same row, column, or diagonal).
- **Path Cost:** Zero or 1 per step (irrelevant in this problem, as we only care about the final configuration, not the path).

Search Strategy: Depth-First Search (DFS) with Backtracking is highly effective for this Constraint Satisfaction Problem (CSP). It places queens one by one, column by column. If placing a queen leads to a conflict, it backtracks, removing the queen and trying the next available row.

Python Implementation

```
def solve_n_queens(n):
    def is_safe(board, row, col):
        # Check this row on left side
        for i in range(col):
            if board[row][i] == 1: return False

        # Check upper diagonal on left side
        for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
            if board[i][j] == 1: return False

        # Check lower diagonal on left side
        for i, j in zip(range(row, n, 1), range(col, -1, -1)):
            if board[i][j] == 1: return False
```

```

        return True

def solve_util(board, col):
    # Base case: All queens placed
    if col >= n: return True

    for i in range(n):
        if is_safe(board, i, col):
            board[i][col] = 1 # Place queen

            # Recur to place rest
            if solve_util(board, col + 1):
                return True

            # Backtrack: Remove queen
            board[i][col] = 0
    return False

board = [[0 for _ in range(n)] for _ in range(n)]
if not solve_util(board, 0):
    print("Solution does not exist")
    return False

print("One possible safe configuration for 8-Queens:")
for row in board:
    print(" ".join(["Q" if x == 1 else "." for x in
row]))
    return True

solve_n_queens(8)

```

Question 4: OLA Cab booking - Agent type and Pseudocode.

Answer

Type of Problem-Solving Agent: A **Goal-based Agent**, specifically employing a *Pathfinding or Search Algorithm (like A* or Dijkstra's)*. The agent must consider a sequence of future actions (turning on streets, avoiding traffic) to achieve a single clear goal: navigating from the starting pickup location to the final destination in the shortest time or distance.

Pseudocode (A* Search Algorithm for Routing):

```
Function Find_Cab_Route(Graph, Start, Destination):
    Initialize PriorityQueue frontier
    frontier.push(Start, priority=0)

    Initialize Dictionary came_from // to reconstruct path
    Initialize Dictionary cost_so_far // tracks optimal costs

    came_from[Start] = Null
    cost_so_far[Start] = 0

    While frontier is not empty:
        current = frontier.pop()

        if current == Destination:
            return Reconstruct_Path(came_from, Destination)

        For each neighbor in Graph.get_neighbors(current):
            new_cost = cost_so_far[current] + Graph.cost(current, neighbor)

            if neighbor not in cost_so_far OR new_cost < cost_so_far[neighbor]:
                cost_so_far[neighbor] = new_cost
                priority = new_cost + Heuristic_Estimate(neighbor, Destination)
                frontier.push(neighbor, priority)
                came_from[neighbor] = current

    Return "No Valid Route Found"
```


Python Implementation

```
import heapq

def ola_route_finder(graph, start, destination):
    # frontier stores tuples of (estimated_total_cost,
    current_cost, node, path)
    frontier = [(0, 0, start, [])]
    visited_costs = {start: 0}

    # Simple heuristic function (e.g., straight-line
    distance, mocked as 0 for Dijkstra)
    def heuristic(node, goal):
        return 0 # Converts A* back to Dijkstra's if no
        coordinates exist

    while frontier:
        est_total, current_cost, current_node, path =
        heapq.heappop(frontier)

        # Update path
        path = path + [current_node]

        if current_node == destination:
            return path, current_cost

        for neighbor, travel_cost in
        graph.get(current_node, {}).items():
            new_cost = current_cost + travel_cost

            if neighbor not in visited_costs or new_cost <
            visited_costs[neighbor]:
                visited_costs[neighbor] = new_cost
                priority = new_cost + heuristic(neighbor,
                destination)
                heapq.heappush(frontier, (priority,
                new_cost, neighbor, path))

    return None, float('inf')

# Mock OLA City Graph
city_map = {
    'Pickup': {'Junction A': 5, 'Junction B': 2},
    'Junction A': {'Destination': 4},
```

```

        'Junction B': {'Junction A': 1, 'Destination': 7},
        'Destination': {}
    }

    route, cost = ola_route_finder(city_map, 'Pickup',
    'Destination')
    print(f"Optimal OLA Cab Route: {' -> '.join(route)}")
    print(f"Total Trip Cost/Time: {cost} mins")

```

Question 5: Logistics company - Uniform Cost Search (UCS) on weighted graph.

Answer

Problem Tracing using UCS:

The Uniform Cost Search explores nodes in order of their cumulative path cost from the start node (S).

- **Step 1:** Start at **S**. Cost = 0. Queue: [(0, S)].
- **Step 2:** Expand **S**. Add neighbors A and G. Queue becomes: [(1, A), (12, G)].
- **Step 3:** Pop lowest cost node **A (Cost: 1)**. Expand A. Add neighbors B and C. Cost to B = 1+3=4. Cost to C = 1+1=2. Queue becomes: [(2, C), (4, B), (12, G)].
- **Step 4:** Pop lowest cost node **C (Cost: 2)**. Expand C. Add neighbors D and G. Cost to D = 2+1=3. Cost to G = 2+2=4. Queue becomes: [(3, D), (4, B), (4, G), (12, G)].
- **Step 5:** Pop lowest cost node **D (Cost: 3)**. Expand D. Add neighbor G. Cost to G via D = 3+3=6. Queue becomes: [(4, B), (4, G), (6, G), (12, G)].
- **Step 6:** Pop **B (Cost: 4)**. Expand B. Add neighbor D. Cost to D via B = 4+3=7 (Ignore, D already found at cost 3).
- **Step 7:** Pop **G (Cost: 4)**. Since G is the destination, we stop!

Result: The least-cost path is **S → A → C → G** with a total minimum cost of **4**.

Python Implementation

```
import heapq

def uniform_cost_search(graph, start, goal):
    # Priority Queue stores tuples of (cumulative_cost,
    current_node, path)
    priority_queue = [(0, start, [])]
    visited = set()

    while priority_queue:
        cost, node, path = heapq.heappop(priority_queue)

        if node in visited:
            continue

        visited.add(node)
        path = path + [node]

        if node == goal:
            return path, cost

        for neighbor, weight in graph.get(node, []):
            if neighbor not in visited:
                heapq.heappush(priority_queue, (cost +
weight, neighbor, path))

    return None, float('inf')

# Graph representation as adjacency list based on the
provided figure
delivery_network = {
    'S': [('A', 1), ('G', 12)],
    'A': [('B', 3), ('C', 1)],
    'B': [('D', 3)],
    'C': [('D', 1), ('G', 2)],
    'D': [('G', 3)],
    'G': []
}

optimal_path, optimal_cost =
uniform_cost_search(delivery_network, 'S', 'G')
print(f"Least-cost path from S to G: {' ->
'.join(optimal_path)}")
```

```
print(f"Total delivery cost: {optimal_cost}")
```